Resource-Constrained PDDL+ Planning for a Battery-Limited Planetary Rover

Endri Gjinaj
Robotics Engineering
University of Genoa
Genoa, Italy

***Abstract*—A planetary rover must coordinate science activities with limited memory, finite energy, travel time, and data that becomes unusable if it is not returned to base in time. This work studies those constraints in a reproducible PDDL+ model. The rover moves over seeded terrain, collects and encodes scientific datasets, stores them in onboard memory, and offloads them at a base station. Movement consumes terrain-dependent energy from a fixed 40-unit battery, while encoding and corruption evolve continuously. The final experiment combines three dataset counts, three memory levels, three corruption levels, and five unseen seeds, for a total of 135 missions. ENHSP with the `sat-hadd` configuration was run with a standard 30-second limit. Three timeouts were rerun only to obtain a final feasibility label. Mission success fell from 100% with two datasets to 60% with four datasets, while it rose from 46.7% at low memory to 100% at high memory. Memory had the strongest association with feasibility, especially for larger missions. Corruption severity did not change final success in the tested ranges. Runtime and battery margin both worsened as the workload increased. The results show that rover feasibility is determined by the interaction between storage, travel, timing, and energy rather than by any single constraint in isolation.**

***Index Terms*—PDDL+, automated planning, planetary rover, resource constraints, ENHSP, ROS 2, reproducible experiments**

I. Introduction

Planetary-rover planning is difficult because several resources evolve together. A science activity may be individually possible, yet the complete mission can still fail when travel time, battery use, onboard storage, and data-validity limits are considered at the same time. Operational systems such as MAPGEN showed that automated planning can support rover teams in building schedules that satisfy temporal and engineering constraints [1], [2]. More recent work has continued to study consumptive resources and flexible execution for planetary missions [3], [4].

PDDL2.1 introduced durative actions and numeric fluents for temporal planning [5]. PDDL+ extended the language with autonomous processes and events, making it suitable for mixed discrete-continuous systems [6]. In the model used here, movement is initiated by an action, travel progress evolves as a process, and arrival is handled by an event. Encoding and data corruption also evolve while the rover is travelling.

The study addresses the following question:

How do onboard memory, mission size, and time-dependent data degradation affect feasibility, generated-plan efficiency, and planning difficulty in a battery-constrained rover mission?

The goal is not to introduce a new planner. Instead, the contribution is a complete experimental workflow: a PDDL+ rover domain, deterministic problem generator, ENHSP runner and parser, controlled final experiment, statistical notebook, and ROS 2 service/client demonstration. Three expectations guided the analysis: larger missions should be harder; additional memory should improve feasibility and reduce unnecessary travel; and corruption limits may affect either feasibility or planner search effort.

II. Related Work

Automated planning has been used in space operations for both schedule construction and onboard autonomy. MAPGEN combined planning, scheduling, and human decision making for the Mars Exploration Rover mission [1]. Bresina *et al.* describe how temporal constraints and oversubscribed resources were handled during daily rover operations [2]. These systems establish the practical need for resource-aware planning but are not designed as controlled sensitivity studies.

Hybrid planning has also been applied directly to autonomous planetary vehicles. Della Penna *et al.* considered energy, time, movement, and safety in a resource-optimal planning model [7]. PDDL+ provides the formal basis for this type of model because it combines actions, continuous processes, and events [6]. Planning algorithms for PDDL+ must handle the resulting search difficulty, including events and numeric evolution [8], [9]. ENHSP uses heuristic techniques for numeric planning, including interval-based relaxations [10]. These works motivate measuring runtime as well as mission success.

The present study focuses on a narrower experimental question: how memory, science workload, and data degradation interact under a fixed battery budget. The full factorial design makes it possible to separate the effect of each factor while also examining the important interaction between mission size and memory.

III. Rover Model and Software Pipeline

A. Mission Model

The map is a bidirectional line containing a base and one science site for each dataset. Every edge receives one of four terrain profiles: easy, moderate, rocky, or steep. Their travel

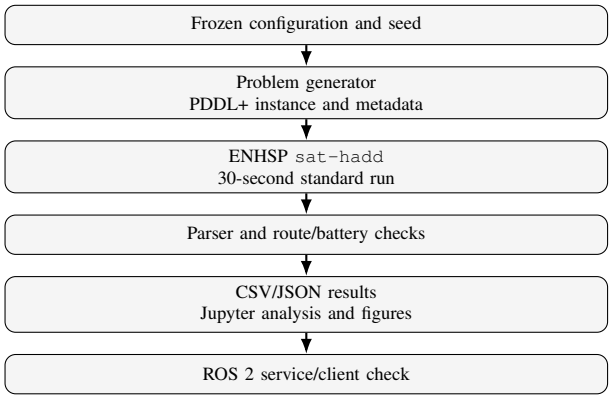


Fig. 1. Experiment and software-integration pipeline.

times are 1.5, 2.0, 2.5, and 3.0 time units, with movement-energy costs of 2, 3, 4, and 5 units. Terrain is generated deterministically from the mission seed.

Movement is represented by a `start-move` action, a continuous `rover-travel` process, and an automatic `arrive` event. The action checks that enough battery remains and subtracts the edge cost. The process then advances travel progress until the arrival event updates the rover position.

Each dataset has a size, encoding duration, corruption value, and loss threshold. Collection occupies memory and starts both encoding and corruption. Encoding completion and data loss are represented by events. A dataset may be offloaded only at the base after encoding is complete; offloading releases its memory. The goal requires every dataset to be offloaded without being lost.

For dataset sizes s_i, memory capacity is defined as

$$M = \max\left(\max_i s_i, \left\lceil r \sum_i s_i \right\rceil\right), \tag{1}$$

where r is 0.45, 0.70, or 1.00 for low, medium, or high memory. The first term ensures that every dataset fits individually. A mission can therefore fail only because of the combined constraints. Battery capacity is fixed at 40 units in every instance.

B. Implementation

Figure 1 shows the main software path. A frozen JSON configuration and seed are passed to a Python generator, which creates a PDDL+ problem and matching metadata. ENHSP is executed with `sat-hadd`; the parser records the planner outcome, runtime, actions, makespan, and temporal information. A validation step reconstructs movement energy and remaining battery. Results are stored in CSV and JSON form and analysed in Jupyter. The same generator and planner runner are exposed through a ROS 2 service.

IV. Experimental Methodology

A. Design

The final study uses a $3 \times 3 \times 3 \times 5$ factorial design. The factors are dataset count, memory level, corruption level, and seed. Table I lists the settings. Dataset sizes range from 2 to 6 units and encoding durations from 2 to 5 time units. Corruption is controlled through positive safety margins: low corruption uses margins from 10 to 14, medium from 6 to 10, and high from 3 to 5. Every dataset is individually safe, but the complete mission may still become unsolvable because of extra trips, memory pressure, battery use, and overlapping deadlines.

TABLE I
Final Experiment Settings

Component	Values
Dataset count	2, 3, 4
Memory ratio	0.45, 0.70, 1.00
Corruption level	low, medium, high
Final seeds	10–14
Terrain travel time	1.5, 2.0, 2.5, 3.0
Terrain energy cost	2, 3, 4, 5
Battery capacity	40
Planner	ENHSP `sat-hadd`
Standard limit	30 s
Instances	135

Seeds 0–4 were used while checking and calibrating the model. The configuration was then frozen before seeds 10–14 were generated. Separate deterministic random streams preserve fair comparisons: changing memory or corruption does not silently regenerate the terrain or dataset properties.

B. Outcomes and Analysis

The main feasibility outcome is the final solved or unsolvable classification. Planner runtime uses only the original 30-second runs. Three timeouts were rerun with a 120-second limit to obtain a classification, but those longer runtimes were not mixed into the runtime analysis.

Secondary measurements include makespan, movement actions, energy used, and battery remaining. Because plan metrics exist only for solved missions, comparisons across memory levels use complete matched conditions in which all three memory variants were solved. Chi-square tests and Cramer's V are used for feasibility, Kruskal–Wallis tests and epsilon-squared for runtime, and Friedman tests with Kendall's W for matched plan metrics. Wilson intervals accompany the observed success percentages. With five final seeds per condition, the statistical results are treated as exploratory rather than as universal estimates.

V. Results

A. Feasibility

The standard runs produced 107 solved instances, 25 unsolvable instances, and three timeouts. The extended checks classified two timeouts as unsolvable and one as solved. The final result was therefore 108 solved missions and 27 unsolvable missions, with no unresolved cases.

Figure 2(a) summarizes success by factor. Success fell from 100% for two datasets to 80% for three and 60% for four. It increased from 46.7% at low memory to 93.3% at medium

memory and 100% at high memory. All three corruption levels produced the same 80% success rate.

The interaction in Fig. 2(b) is more informative than the marginal percentages alone. Memory had no effect on two-dataset missions because every case was solved. With three datasets, low memory solved only 40% of the missions, while medium and high memory solved all of them. With four datasets, low memory solved none, medium memory solved 80%, and high memory solved all missions.

Dataset count was associated with feasibility, $\chi^2(2) = 22.50$, $p = 1.30 \times 10^{-5}$, with Cramer's $V = 0.408$. Memory showed the stronger association, $\chi^2(2) = 47.50$, $p = 4.85 \times 10^{-11}$, $V = 0.593$. Corruption level showed no association in this sample, $\chi^2(2) = 0$, $p = 1$.

B. Runtime and Battery Margin

Figure 3(a) compares the median and the 90th percentile of the standard runtime. The median rose from 0.384 seconds for two datasets to 0.528 seconds for three and 7.315 seconds for four. The 90th percentile rose from 0.449 to 1.755 and then 25.345 seconds. All three standard timeouts occurred in four-dataset cases. The dataset-count effect was large, $H(2) = 100.99$, $p = 1.17 \times 10^{-22}$, $\epsilon^2 = 0.750$.

Corruption-level means differed descriptively, but the overall runtime test did not reach the 0.05 level, $H(2) = 4.91$, $p = 0.0858$. For this reason, corruption runtime is reported in the notebook table rather than promoted to a separate main figure.

Figure 3(b) shows the battery left after plans found within the standard limit. Solved two-dataset missions retained an average of 21.73 units. The average fell to 8.44 units for three datasets and 1.70 units for four. The four-dataset plans that were found therefore operated close to the fixed 40-unit battery limit.

C. Generated-Plan Efficiency

Figure 4 shows the relationship between movement energy and generated-plan makespan for the 107 plans found within the standard limit. Plans that used more movement energy generally also took longer because both quantities increase with travel. The plot is descriptive: the planner is satisficing, not optimal, and the solved-only sample excludes difficult cases where no plan was found.

For a fair memory comparison, 21 matched mission conditions were selected in which low, medium, and high memory all produced a plan. The mean movement counts were 7.71, 7.14, and 5.33; mean energy use was 24.57, 22.86, and 16.38; and mean makespan was 25.76, 23.52, and 17.10. Friedman tests found large memory effects for movement and energy, $\chi^2_F(2) = 35.29$, $p = 2.17 \times 10^{-8}$, $W = 0.840$, and for makespan, $\chi^2_F(2) = 34.24$, $p = 3.67 \times 10^{-8}$, $W = 0.815$.

D. ROS 2 Check

The same generator and planner runner used by the batch experiment are exposed through a ROS 2 service. In the executed notebook, a request for a three-dataset, medium-memory, medium-corruption mission returned a solved plan with makespan 28, ten movement actions, and 26 battery units used. The remaining 14 units matched the value reconstructed independently in the notebook. This check confirms that the ROS interface returns the same metrics as the offline experiment; it is not counted as an additional statistical trial.

VI. Discussion and Limitations

The clearest result is the interaction between mission size and memory. Small missions do not reveal a storage problem because every memory level is sufficient. As the number of datasets grows, low memory forces more returns to base. Those trips use energy and add time during which collected data can continue to degrade. The four-dataset condition makes the effect visible: low memory fails every mission, while high memory solves every mission.

The corruption result is more limited. The chosen margins guaranteed that each dataset was individually safe, and the planner was usually able to adapt the route within the tested ranges. Corruption therefore did not change final feasibility. Low-corruption cases had a heavier descriptive runtime tail, possibly because wider time windows permit more candidate schedules, but the overall runtime test was not significant. That interpretation should remain tentative.

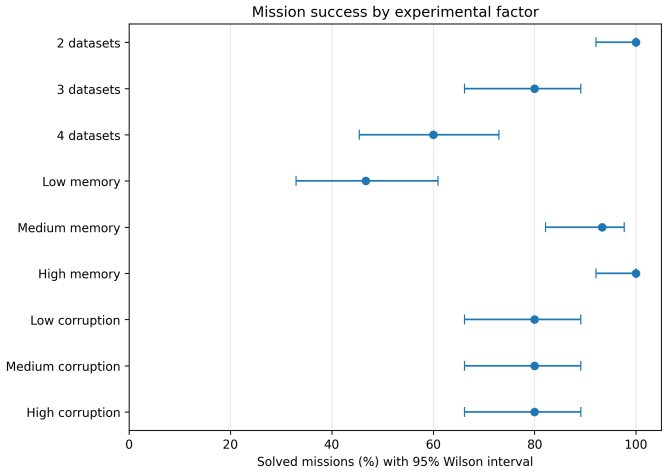
The design supports reproducibility through deterministic generation, frozen parameters, unseen final seeds, and a clear separation between standard runtime and extended feasibility classification. The notebook also checks route and battery consistency for solved plans. These choices improve internal validity, but they do not make the model a complete representation of a flight rover.

Several simplifications limit generalization. The map is linear, terrain is synthetic, corruption is deterministic, and battery is spent only on movement. Real systems also consume energy for sensing, computation, communication, thermal control, and standby. The experiment uses one planner configuration and five final seeds per condition. Wall-clock runtime includes Java and planner startup on the host machine. In addition, `sat-hadd` is satisficing, so reported energy and makespan values are properties of the generated plans rather than guaranteed global optima.

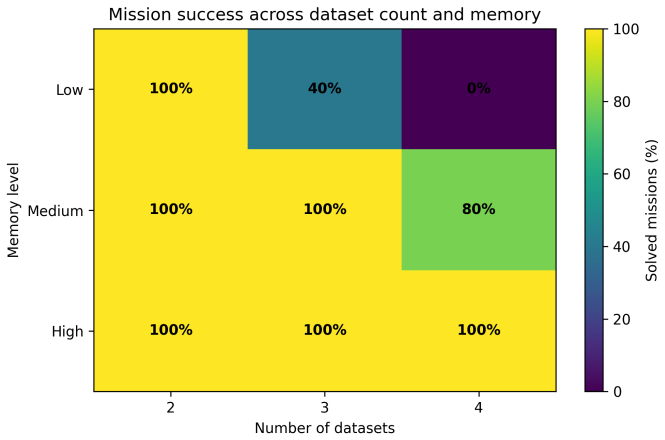
Future work could add alternative routes, communication windows, solar charging, probabilistic degradation, additional energy consumers, and comparisons across planners or heuristics. A larger experiment would also help determine whether the observed corruption-runtime pattern is stable.

VII. Conclusion

The final experiment evaluated 135 battery-constrained rover missions while varying science workload, onboard memory, and data degradation. Larger missions were less feasible, took longer to plan, and left less battery after successful execution. Memory was the strongest feasibility factor, but its importance appeared mainly when the workload increased. Higher memory also reduced movement, energy use, and makespan in matched solved missions. Corruption did not affect final success within the selected positive-margin ranges.

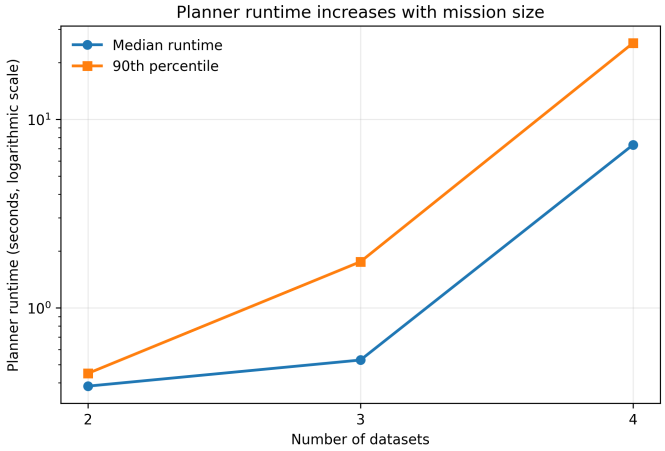


(a) Success by factor with 95% Wilson intervals.

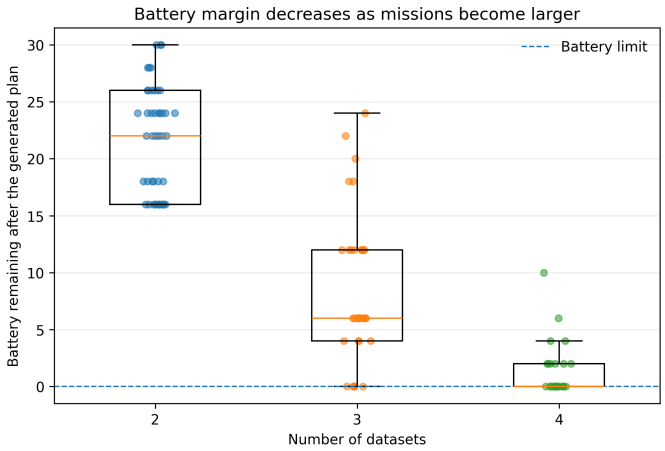


(b) Interaction between mission size and memory.

Fig. 2. Final mission feasibility. Each marginal point contains 45 missions, and each heatmap cell contains 15 missions.



(a) Median and 90th-percentile runtime.



(b) Battery remaining after solved missions.

Fig. 3. Planning cost and battery margin as the number of datasets increases. Runtime uses only the original 30-second runs.

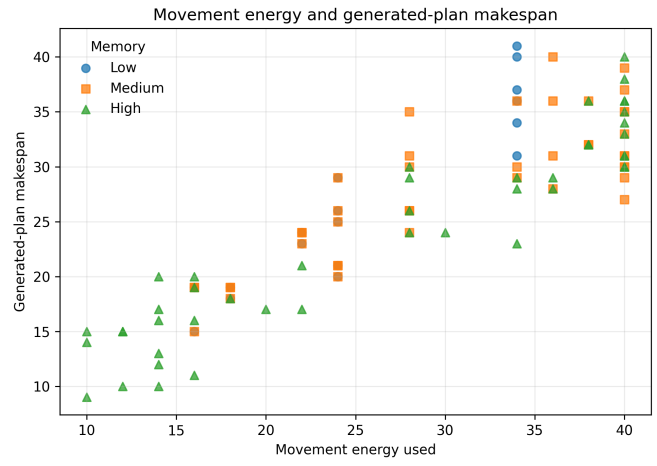


Fig. 4. Movement energy and makespan for plans found within the standard 30-second limit. Marker shape indicates the memory level.

The study shows why rover resources should be analysed together. Storage decisions change travel requirements; travel changes both energy use and timing; and those changes determine whether all data can be returned safely. The repository provides the frozen model, generator, final results, executed notebook, statistical tables, figures, and ROS 2 interface needed to reproduce and extend the analysis.

References

[1] M. Ai-Chang et al., "MAPGEN: Mixed-initiative planning and scheduling for the mars exploration rover mission," *IEEE Intelligent Systems*, vol. 19, no. 1, pp. 8–12, 2004.

[2] J. L. Bresina, A. K. Jónsson, P. H. Morris, and K. Rajan, "Activity planning for the mars exploration rovers," in *Proceedings of the 15th International Conference on Automated Planning and Scheduling*, 2005, pp. 40–49.

[3] W. Chi, S. Chien, and J. Agrawal, "Scheduling with complex consumptive resources for a planetary rover," in *Proceedings of the Thirtieth International Conference on Automated Planning and Scheduling*, 2020, pp. 348–356.

[4] J. Agrawal, W. Chi, S. A. Chien, G. Rabideau, D. Gaines, and S. Kuhn, "Analyzing the effectiveness of rescheduling and flexible execution methods to address uncertainty in execution duration for a planetary rover," *Robotics and Autonomous Systems*, vol. 140, p. 103 758, 2021. DOI: 10.1016/j.robot.2021.103758.

[5] M. Fox and D. Long, "PDDL2.1: An extension to PDDL for expressing temporal planning domains," *Journal of Artificial Intelligence Research*, vol. 20, pp. 61–124, 2003. DOI: 10.1613/jair.1129.

[6] M. Fox and D. Long, "Modelling mixed discrete–continuous domains for planning," *Journal of Artificial Intelligence Research*, vol. 27, pp. 235–297, 2006. DOI: 10.1613/jair.2044.

[7] G. D. Penna, B. Intrigila, D. Magazzeni, and F. Mercorio, "Resource-optimal planning for an autonomous planetary vehicle," *International Journal of Artificial Intelligence and Applications*, vol. 1, no. 3, pp. 15–29, 2010. DOI: 10.5121/ijaia.2010.1302.

[8] A. J. Coles and A. I. Coles, "PDDL+ planning with events and linear processes," in *Proceedings of the 24th International Conference on Automated Planning and Scheduling*, 2014, pp. 74–82. DOI: 10.1609/icaps.v24i1.13647.

[9] W. Piotrowski, M. Fox, D. Long, D. Magazzeni, and F. Mercorio, "Heuristic planning for PDDL+ domains," in *Proceedings of the 25th International Joint Conference on Artificial Intelligence*, 2016, pp. 3213–3219.

[10] E. Scala, P. Haslum, S. Thiébaux, and M. Ramírez, "Interval-based relaxation for general numeric planning," in *Proceedings of the 22nd European Conference on Artificial Intelligence*, IOS Press, 2016, pp. 655–663. DOI: 10.3233/978-1-61499-672-9-655.